ASIAN INSTITUTE OF MANAGEMENT

Machine Learning 3

# Convolutional Neural Networks

## Sessions 5-6

MSDS 2026

23 April 2026

ASIAN INSTITUTE OF MANAGEMENT

# Simple CNN

# Simple CNN: Intro to Keras Example

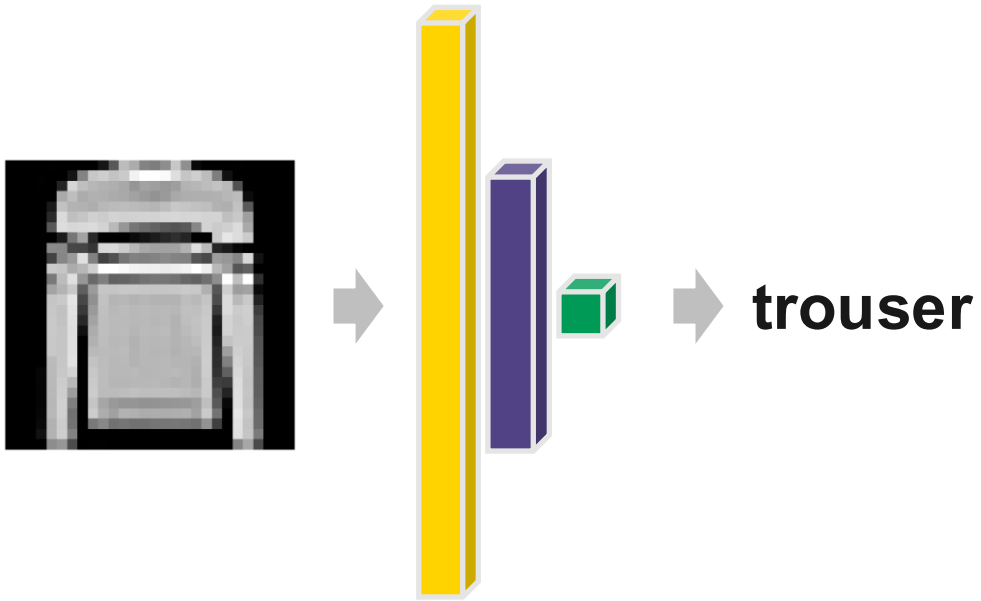
## Clothing Classification using Fashion MNIST Dataset

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten (Flatten) | (None, 784) | 0 |
| dense (Dense) | (None, 256) | 200,960 |
| dense_1 (Dense) | (None, 10) | 2,570 |

**Total params:** 203,530 (795.04 KB)

**Trainable params:** 203,530 (795.04 KB)

**Non-trainable params:** 0 (0.00 B)




Flatten layer
Dense layer
Output layer

# Simple CNN: Intro to Keras Example

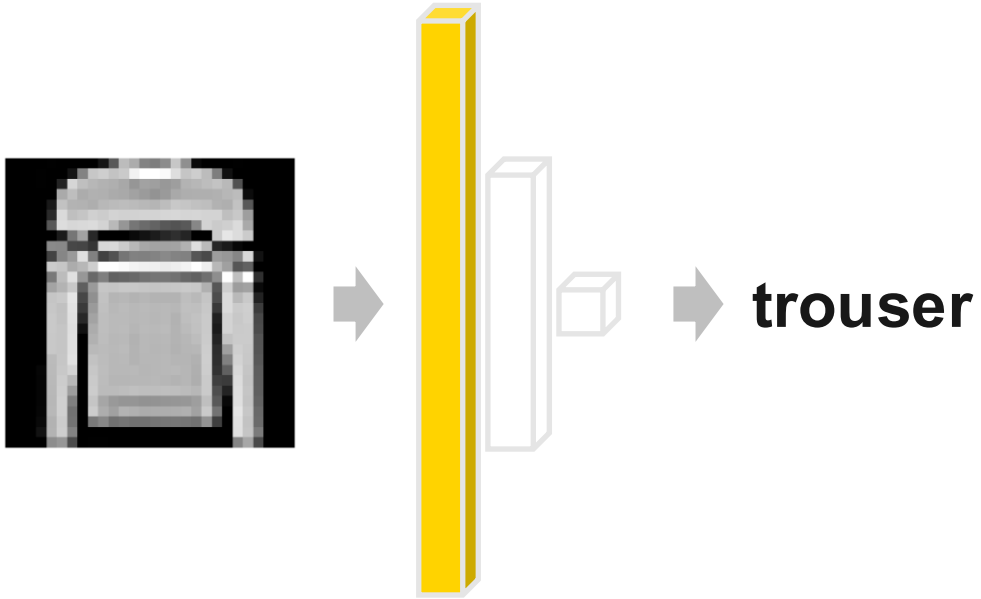
## Clothing Classification using Fashion MNIST Dataset

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten (Flatten) | (None, 784) | 0 |
| dense (Dense) | (None, 256) | 200,960 |
| dense_1 (Dense) | (None, 10) | 2,570 |

Total params: 203,530 (795.04 KB)

Trainable params: 203,530 (795.04 KB)

Non-trainable params: 0 (0.00 B)




784 input neurons

Flatten layer
Dense layer
Output layer

# Simple CNN: Intro to Keras Example

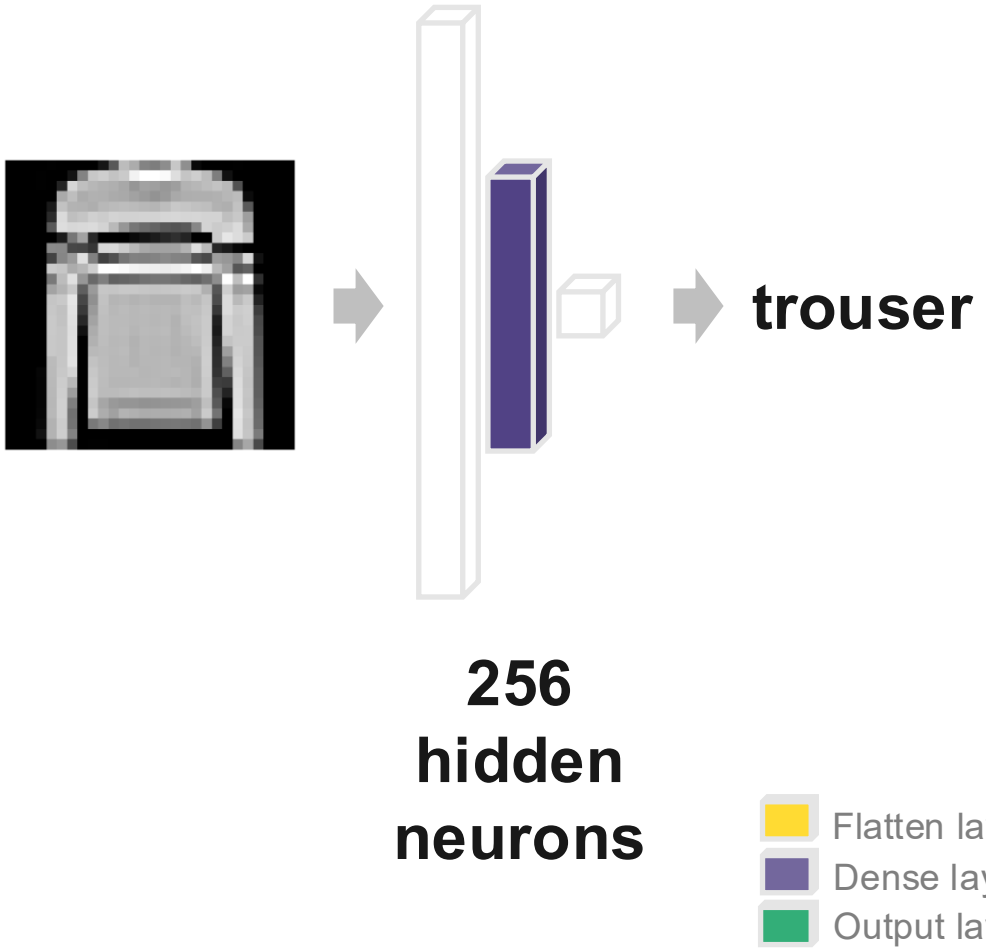
## Clothing Classification using Fashion MNIST Dataset

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten (Flatten) | (None, 784) | 0 |
| dense (Dense) | (None, 256) | 200,960 |
| dense_1 (Dense) | (None, 10) | 2,570 |

Total params: 203,530 (795.04 KB)

Trainable params: 203,530 (795.04 KB)

Non-trainable params: 0 (0.00 B)



# Simple CNN: Intro to Keras Example

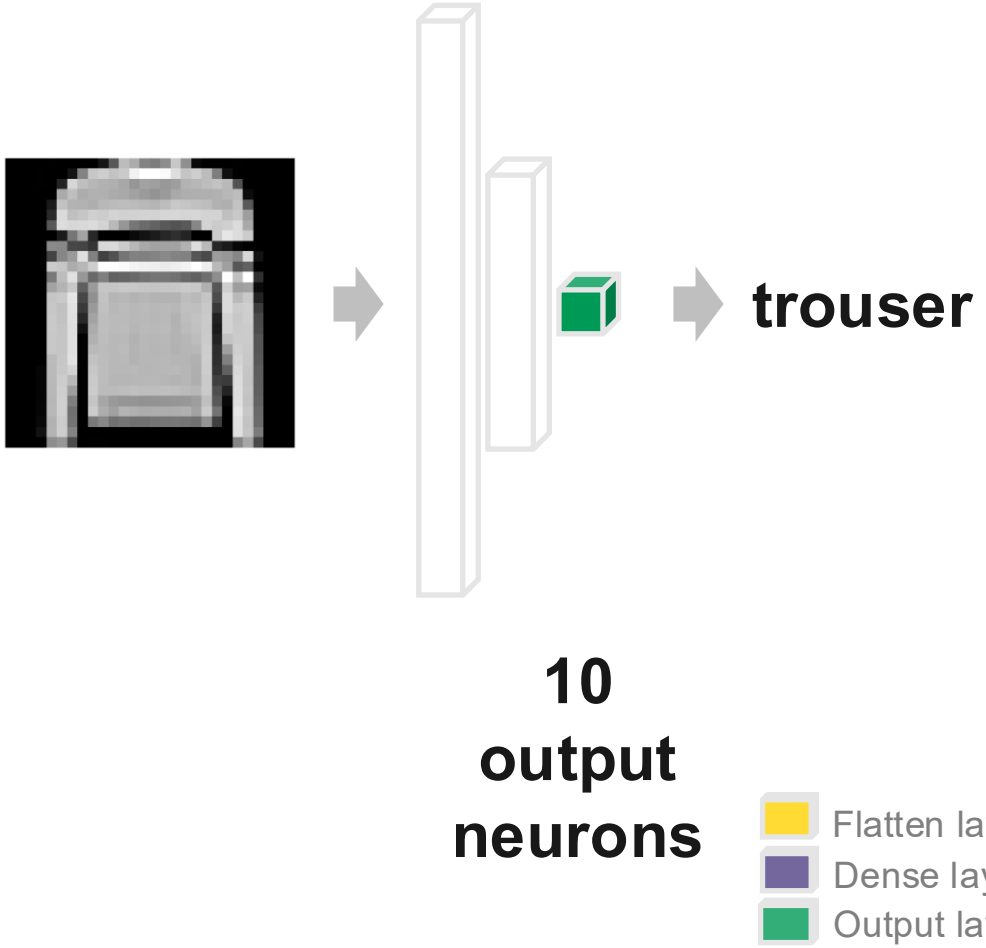
## Clothing Classification using Fashion MNIST Dataset

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten (Flatten) | (None, 784) | 0 |
| dense (Dense) | (None, 256) | 200,960 |
| dense_1 (Dense) | (None, 10) | 2,570 |

**Total params:** 203,530 (795.04 KB)

**Trainable params:** 203,530 (795.04 KB)

**Non-trainable params:** 0 (0.00 B)



# Simple CNN: Intro to Keras Example

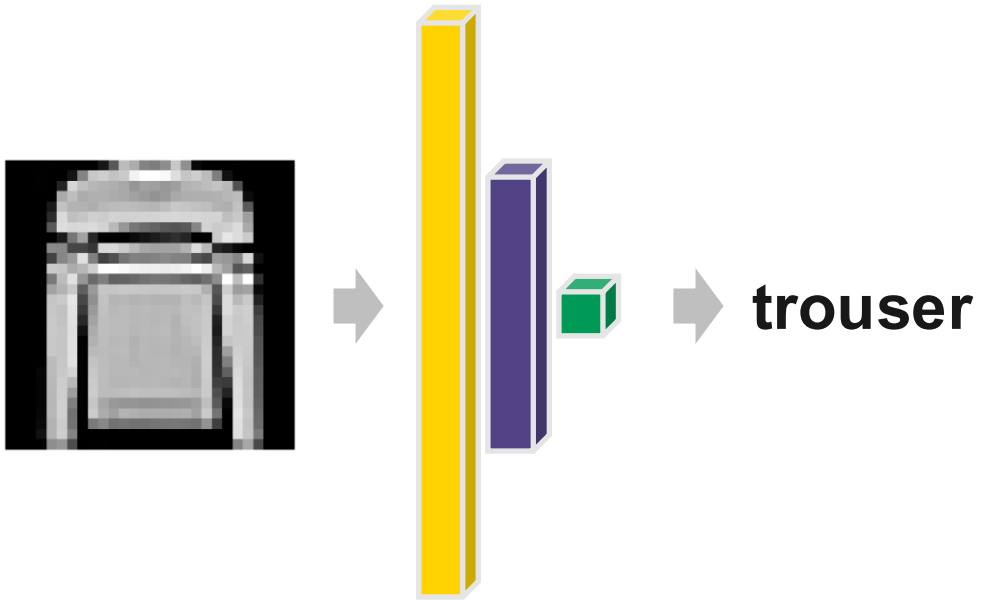
## Clothing Classification using Fashion MNIST Dataset

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten (Flatten) | (None, 784) | 0 |
| dense (Dense) | (None, 256) | 200,960 |
| dense_1 (Dense) | (None, 10) | 2,570 |

**Total params:** 203,530 (795.04 KB)

**Trainable params:** 203,530 (795.04 KB)

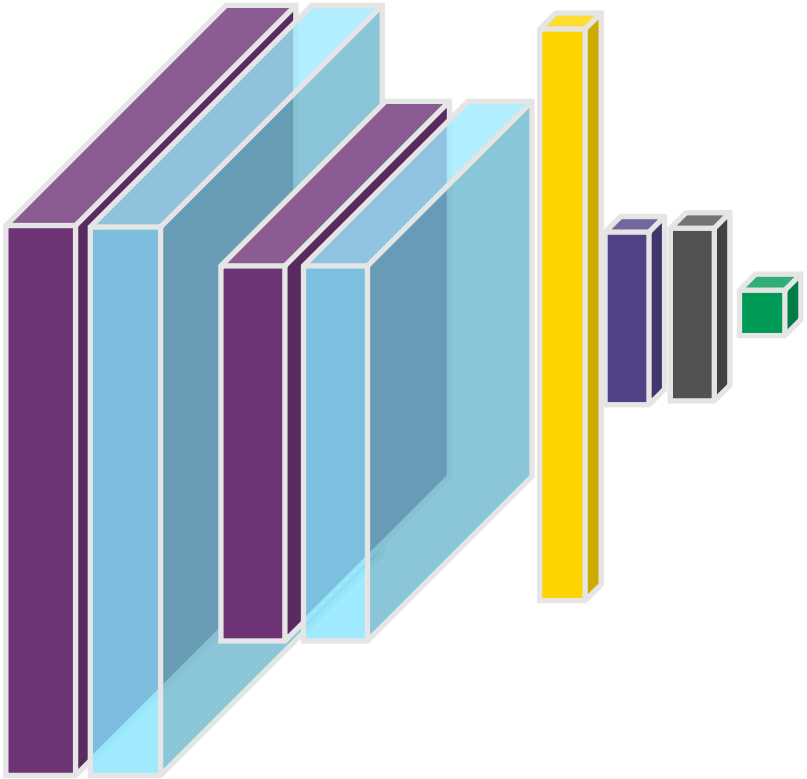
**Non-trainable params:** 0 (0.00 B)




Flatten layer
Dense layer
Output layer

# Simple CNN Example

## Handwritten Digits Classification



| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

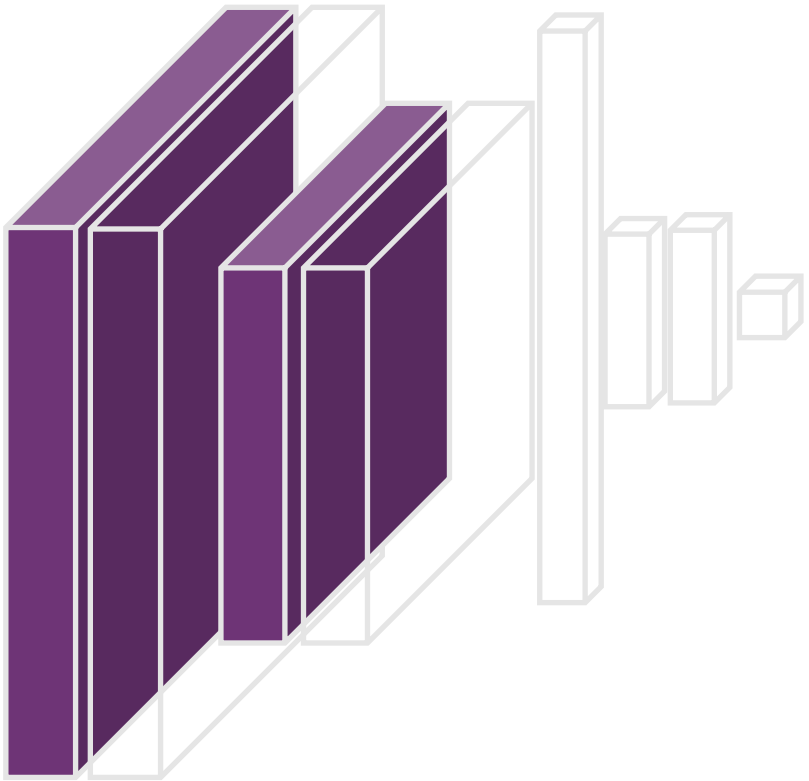
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

Conv2D layer with ReLU activation
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

# Simple CNN Example

## Handwritten Digits Classification



| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

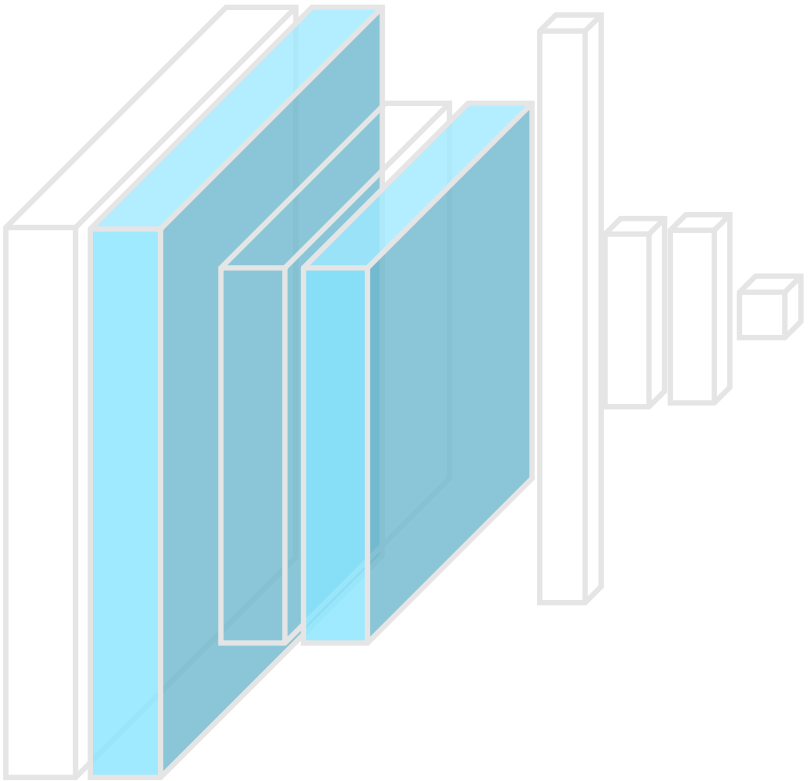
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

Conv2D layer with ReLU activation
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

# Simple CNN Example

## Handwritten Digits Classification



| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

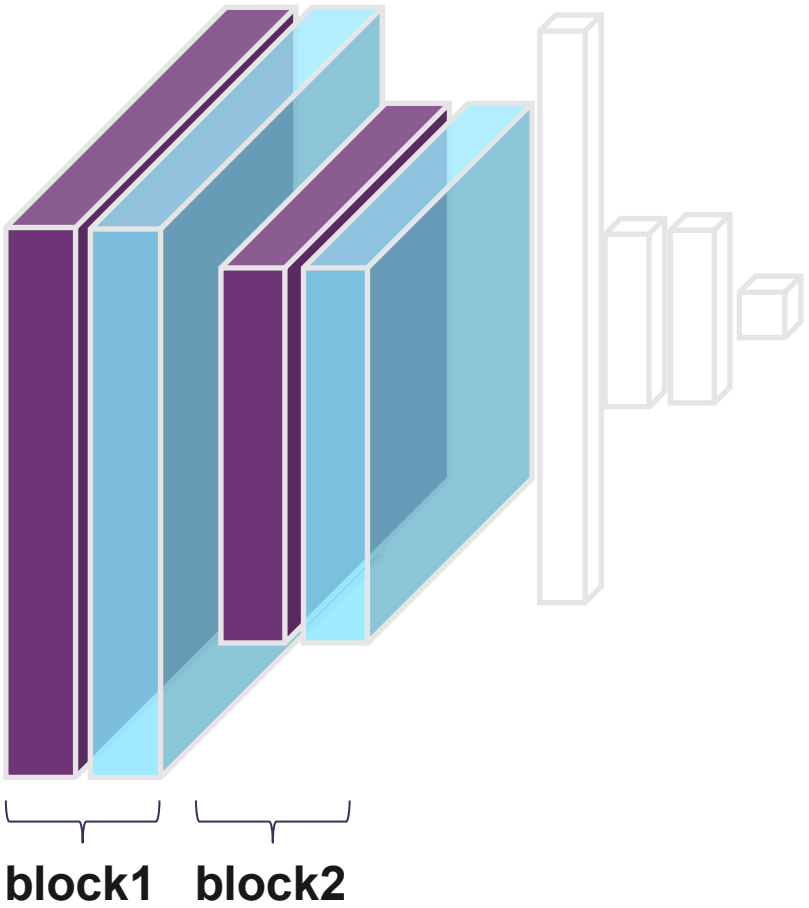
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

Conv2D layer with ReLU activation
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

# Simple CNN Example

## Handwritten Digits Classification




| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

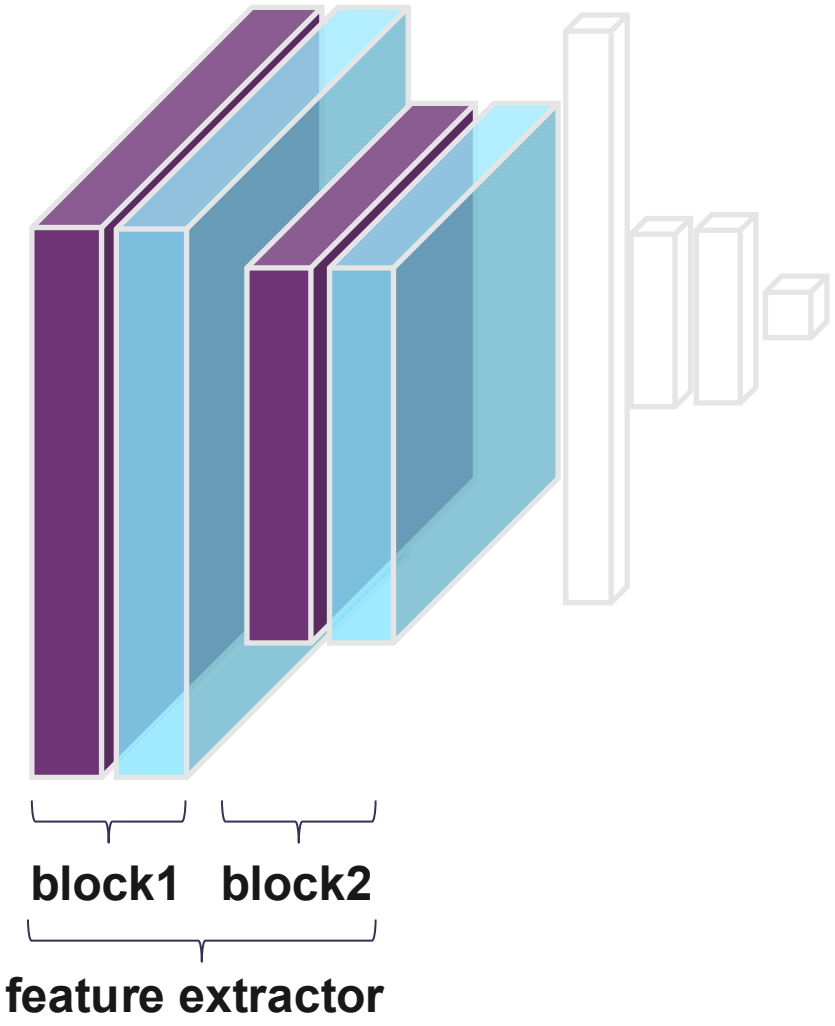
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

- Conv2D layer with ReLU activation
- Max pooling layer
- Dropout layer
- Flatten layer
- Dense layer
- Output layer

# Simple CNN Example

## Handwritten Digits Classification




| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

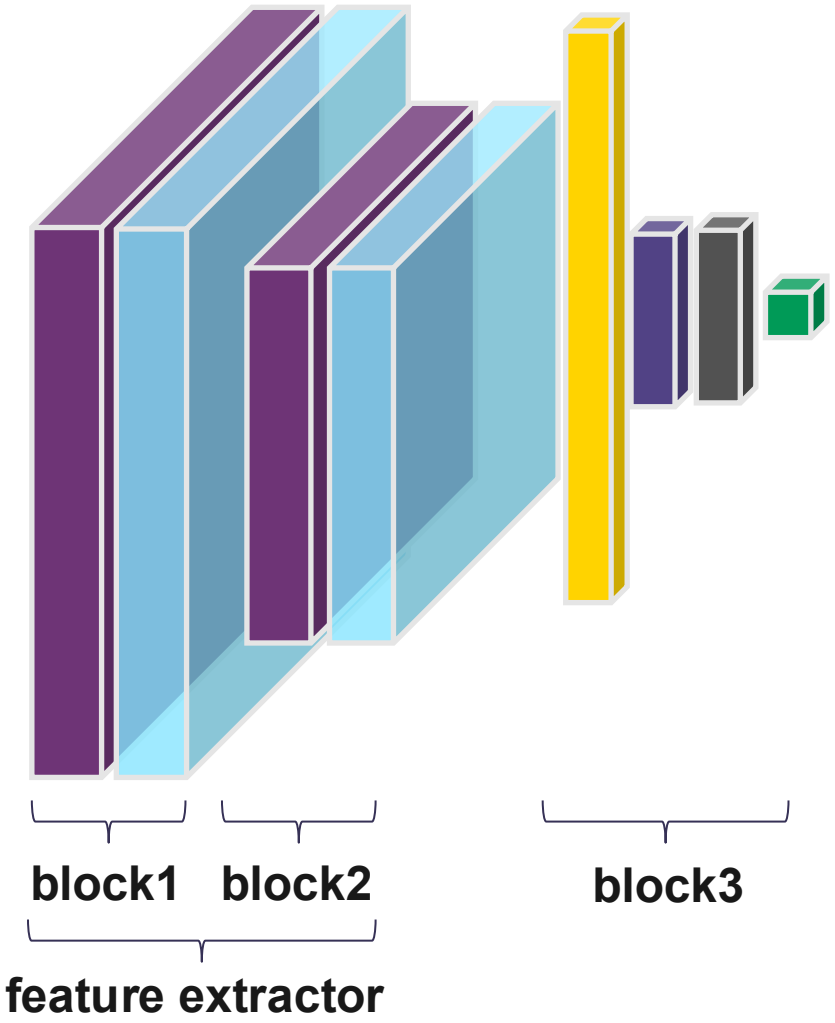
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

- Conv2D layer with ReLU activation
- Max pooling layer
- Dropout layer
- Flatten layer
- Dense layer
- Output layer

# Simple CNN Example

## Handwritten Digits Classification




| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

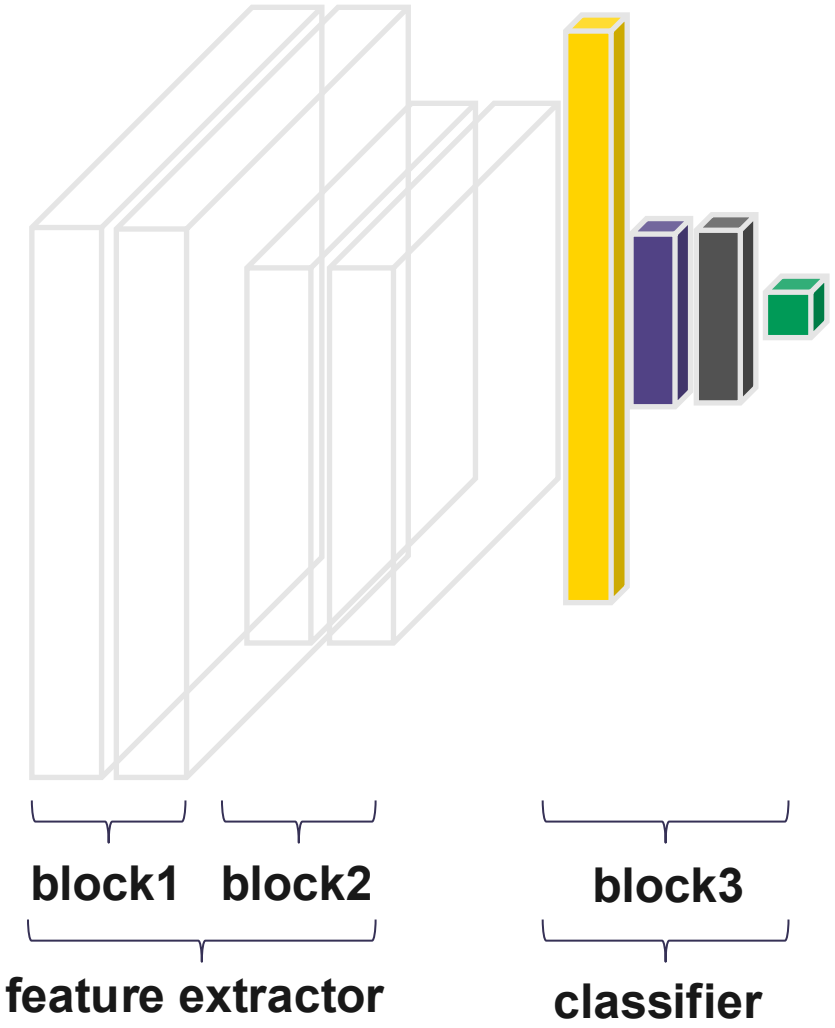
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

- Conv2D layer with ReLU activation
- Max pooling layer
- Dropout layer
- Flatten layer
- Dense layer
- Output layer

# Simple CNN Example

## Handwritten Digits Classification




| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

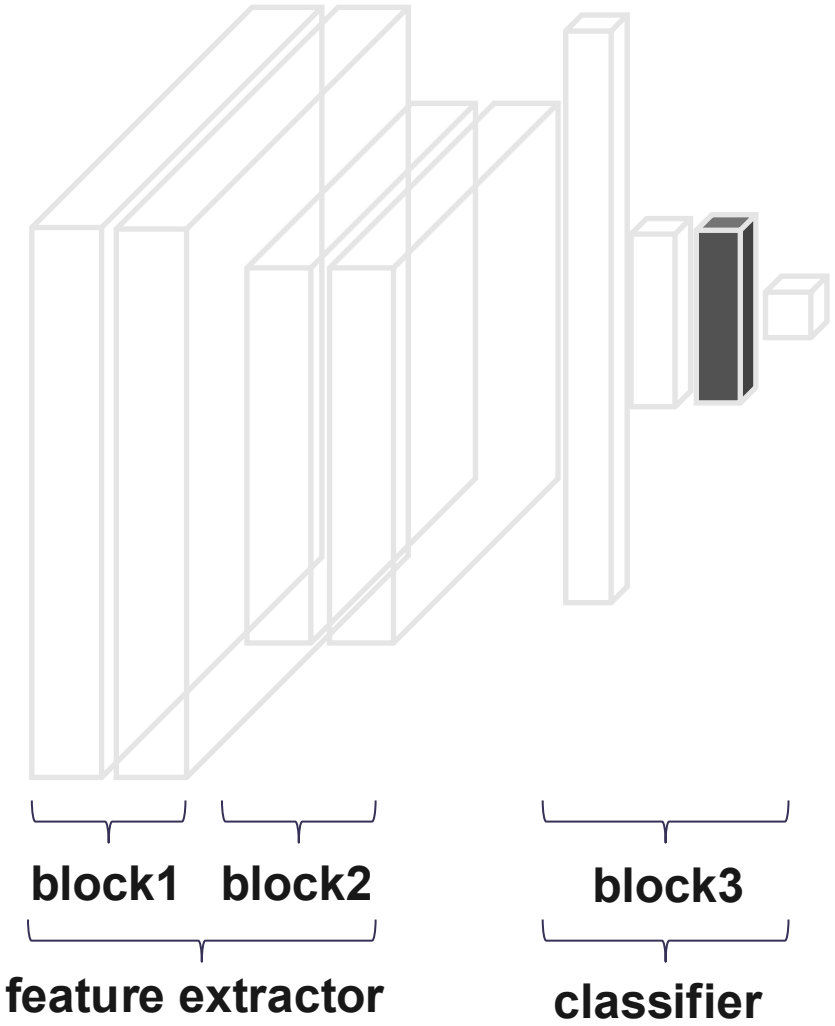
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

Conv2D layer with ReLU activation
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

# Simple CNN Example

## Handwritten Digits Classification




| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

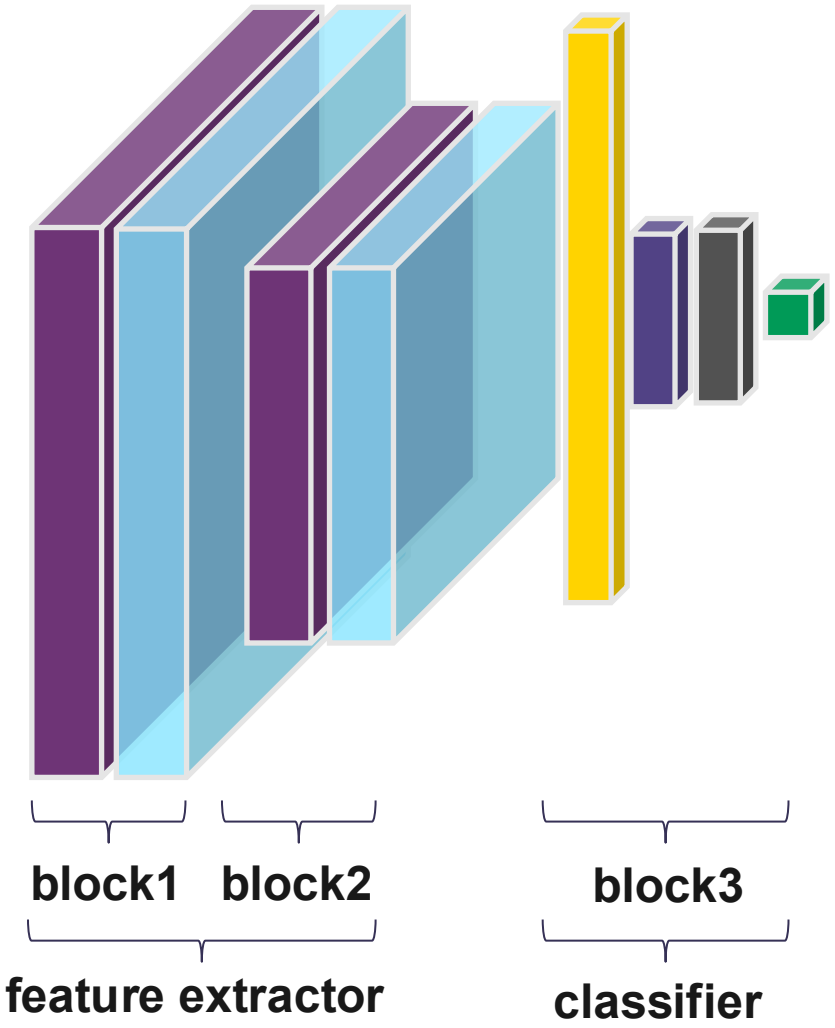
**Trainable params:** 121,930 (476.29 KB)

**Non-trainable params:** 0 (0.00 B)

Conv2D layer with ReLU activation
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

# Simple CNN Example

## Handwritten Digits Classification




| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | 0 |
| flatten (Flatten) | (None, 1600) | 0 |
| dense (Dense) | (None, 64) | 102,464 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 10) | 650 |

**Total params:** 121,930 (476.29 KB)

**Trainable params:** 121,930 (476.29 KB)

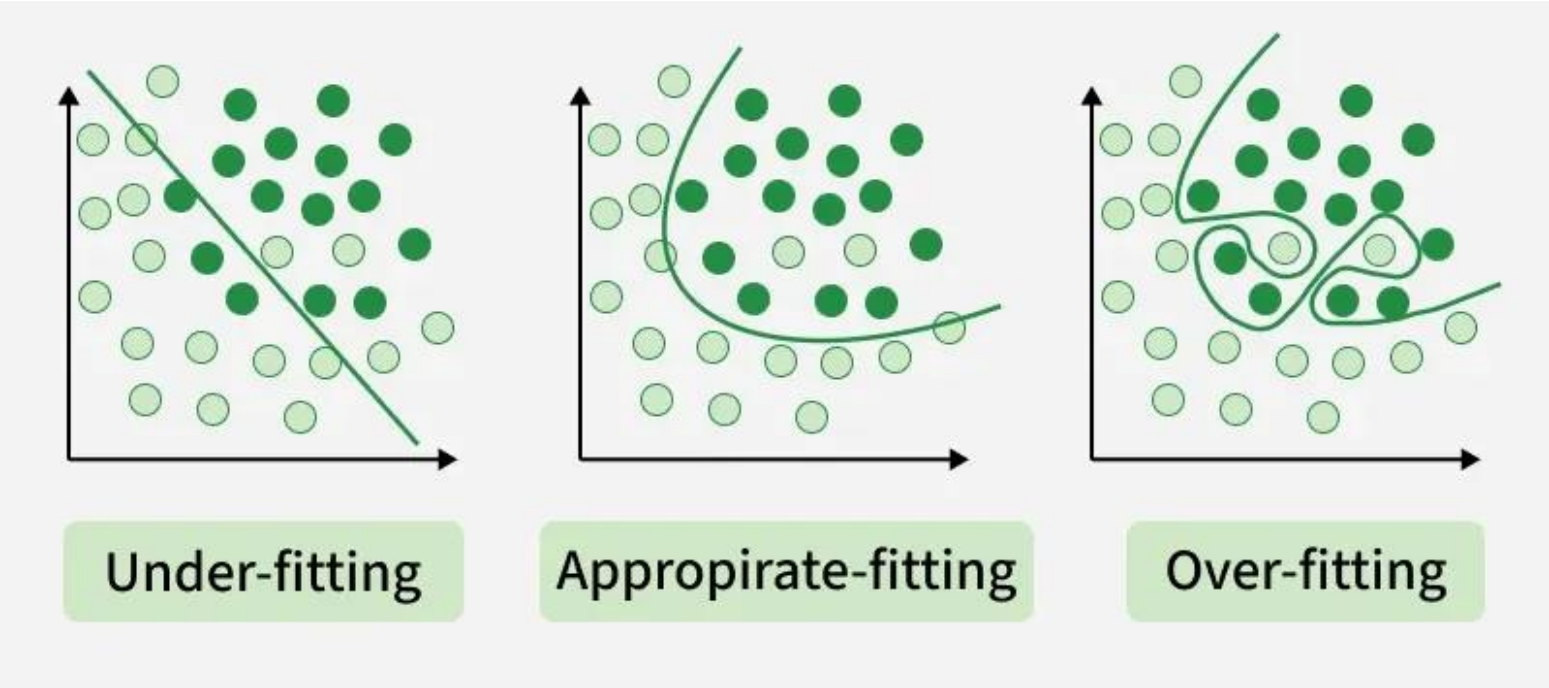
**Non-trainable params:** 0 (0.00 B)

- Conv2D layer with ReLU activation
- Max pooling layer
- Dropout layer
- Flatten layer
- Dense layer
- Output layer

# Regularization

# Regularization

## ML1 and ML2




https://media.geeksforgeeks.org/wp-content/uploads/20251209153305862042/420046945.webp

# Regularization

## Techniques

- Modifying the model architecture
- Altering the training process
- Data augmentation

# Regularization

## Techniques

- Modifying the model architecture
- Altering the training process
- Data augmentation

# Regularization

## Modifying the model architecture

- Applying layer weight regularizers
- Applying dropout
- Applying batch normalization
- Changing the convolution and fully connected layers/blocks



Chollet, F., & Watson, M. (2026). *Deep Learning with Python Third Edition.* Manning.

# Regularization

## Modifying the model architecture

- Applying layer weight regularizers
  - kernel
  - bias
  - output

# Regularization

## Modifying the model architecture

- Applying layer weight regularizers
  - kernel
  - bias
  - output

```
layers.Conv2D(32, (3, 3),
              activation='relu',
              kernel_regularizer=regularizers.L1(0.01),
              bias_regularizer=regularizers.L1(0.01),
              activity_regularizer=regularizers.L2(0.01))
```

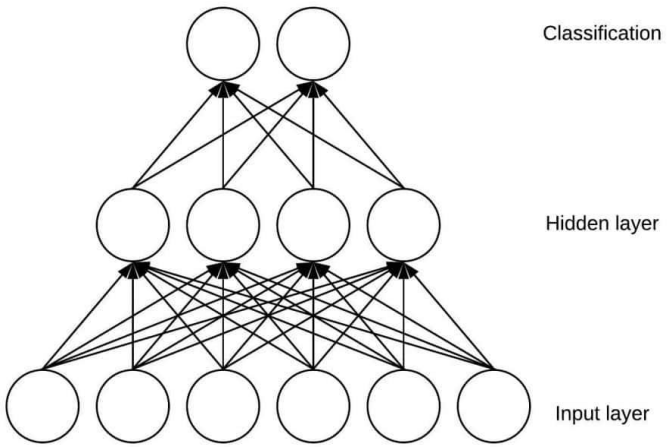


Chollet, F., & Watson, M. (2026). *Deep Learning with Python Third Edition.* Manning.

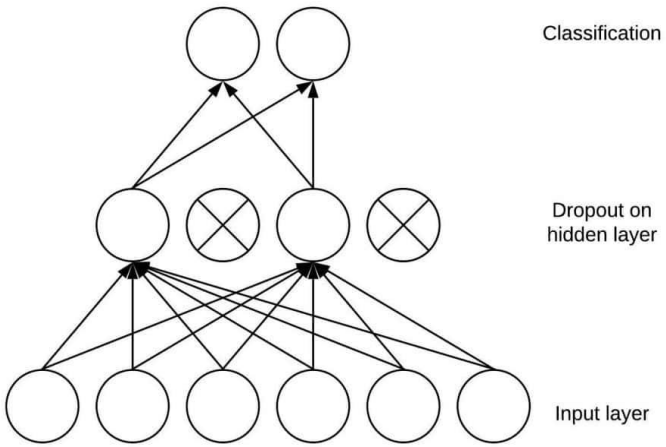
# Regularization

## Modifying the model architecture

- Applying dropout




**Without Dropout**




**With Dropout**

Chollet, F., & Watson, M. (2026). *Deep Learning with Python Third Edition.* Manning.

# Regularization

## Modifying the model architecture

- Applying batch normalization
  - rescale the inputs to the activations of the network so that minibatches have a certain desirable distribution

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift (2015)
https://arxiv.org/abs/1502.03167

# Regularization

## Modifying the model architecture

- Changing the convolution and fully connected layers/blocks
  - Add/delete layers
  - Add/delete convolution blocks

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift (2015)
https://arxiv.org/abs/1502.03167

# Regularization

## Techniques

- Modifying the model architecture
- Altering the training process
- Data augmentation

# Regularization

## Altering the training process

- **Callbacks**
  - an object that can perform actions at various stages of training (e.g. at the start or end of an epoch, before or after a single batch, etc.)

# Regularization

## Altering the training process

- **EarlyStopping**
  - interrupt training as soon as validation metrics have stopped improving, while remembering the best known model state

# Regularization

## Altering the training process

- **ReduceLROnPlateau**
  - reduces learning rate when a metric has stopped improving
- **LearningRateScheduler**
  - updates the optimizer's learning rate using the schedule function based on the current epoch and previous learning rate, at the start of each epoch

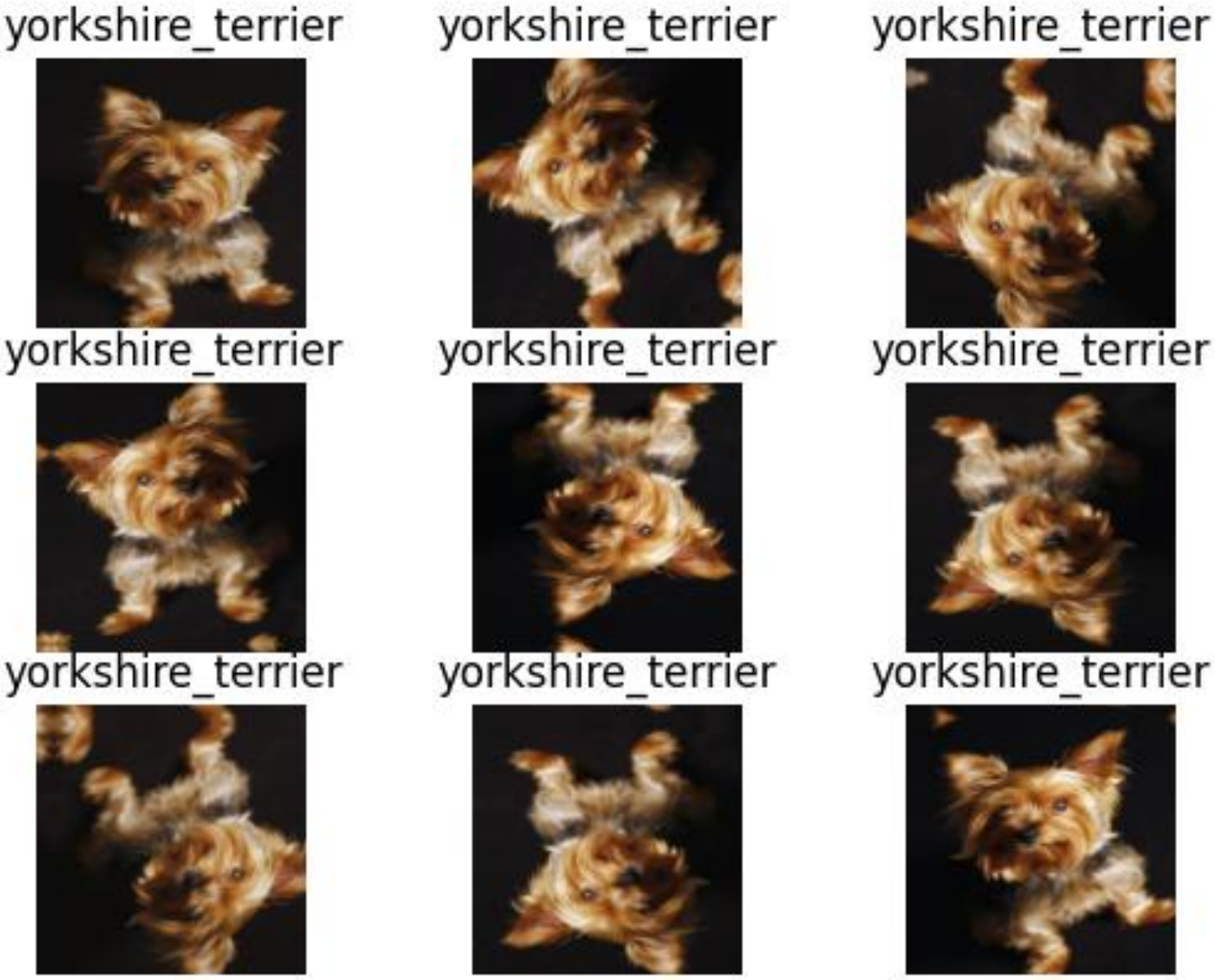
# Regularization

## Techniques

- Modifying the model architecture
- Altering the training process
- Data augmentation

# Regularization

## Data augmentation



# Regularization

## Data augmentation

- artificially introduces sample diversity by applying random yet realistic transformations to the training images
- helps expose the model to different aspects of the training data while slowing down overfitting



Chollet, F., & Watson, M. (2026). *Deep Learning with Python Third Edition.* Manning.

# Regularization

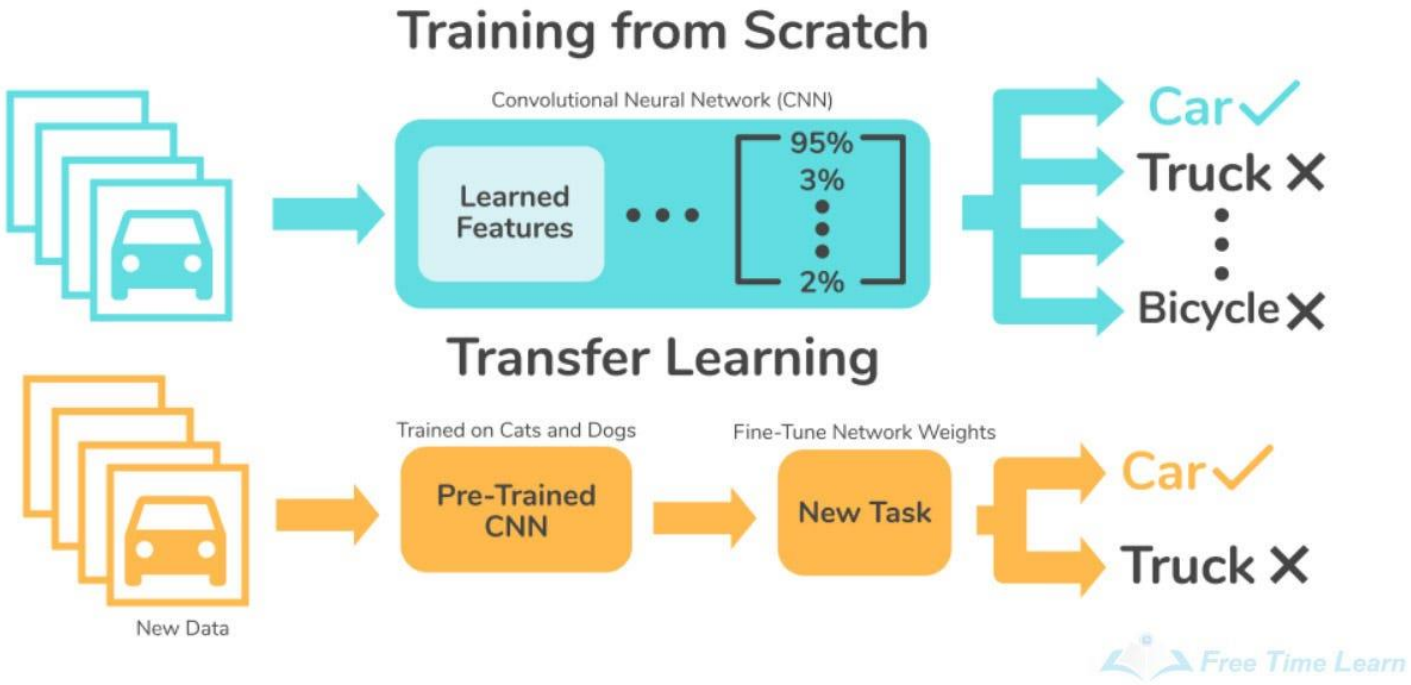
## Data augmentation

- Geometric or spatial transformations
- Color transformations
- Image filtering
- Occlusion

# Transfer Learning

# Regularization

## Transfer learning




https://miro.medium.com/v2/resize:fit:1400/format:webp/1*rInXt5nkZfPL7H95pS5QPA.jpeg

# Transfer learning

## Pretrained model

- a model that was previously trained on a large dataset, typically on a large-scale image-classification task

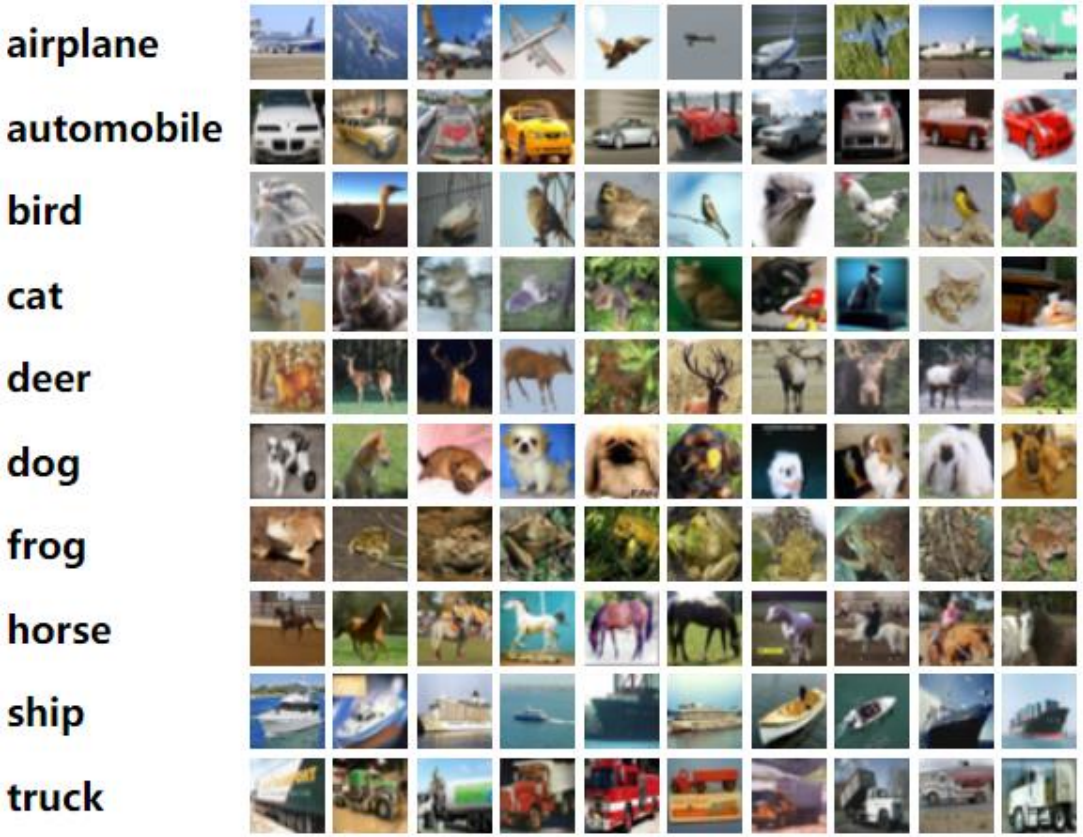
# Transfer learning

## Pretrained model

- **a model that was previously trained on a large dataset, typically on a large-scale image-classification task**
- *Datasets*
  - ImageNet: 14,197,122 images with 1000 classes
  - CIFAR-10 : 60,000 images with 10 classes

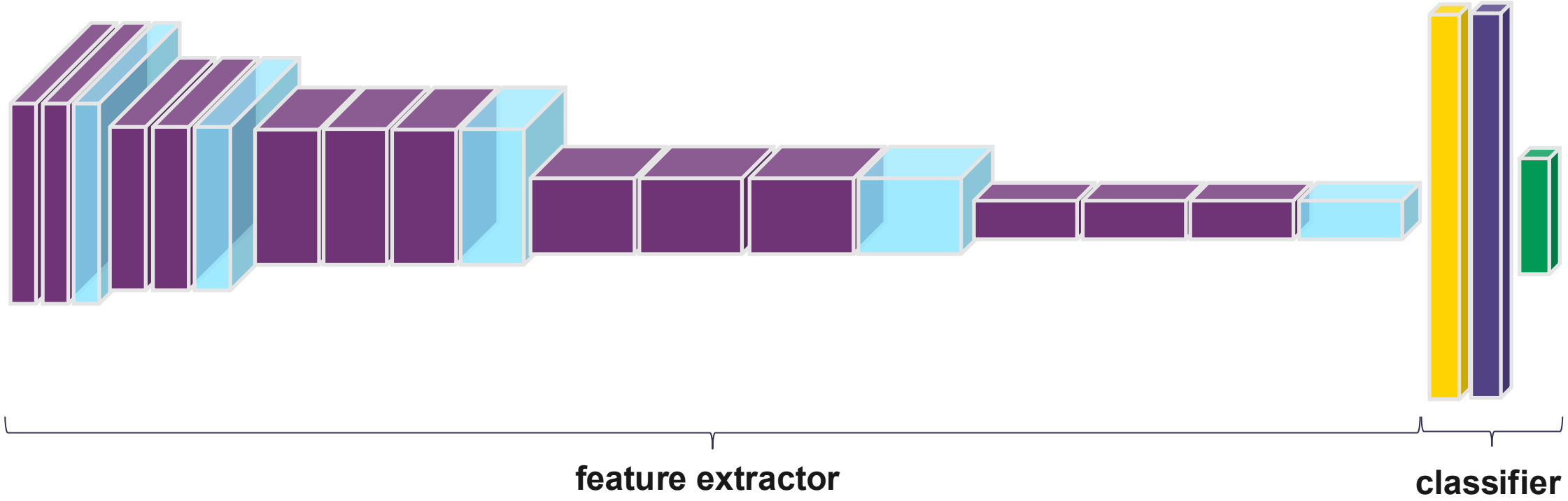
# Transfer learning

## Pretrained model



# Transfer learning

## Pretrained model example - VGG-16

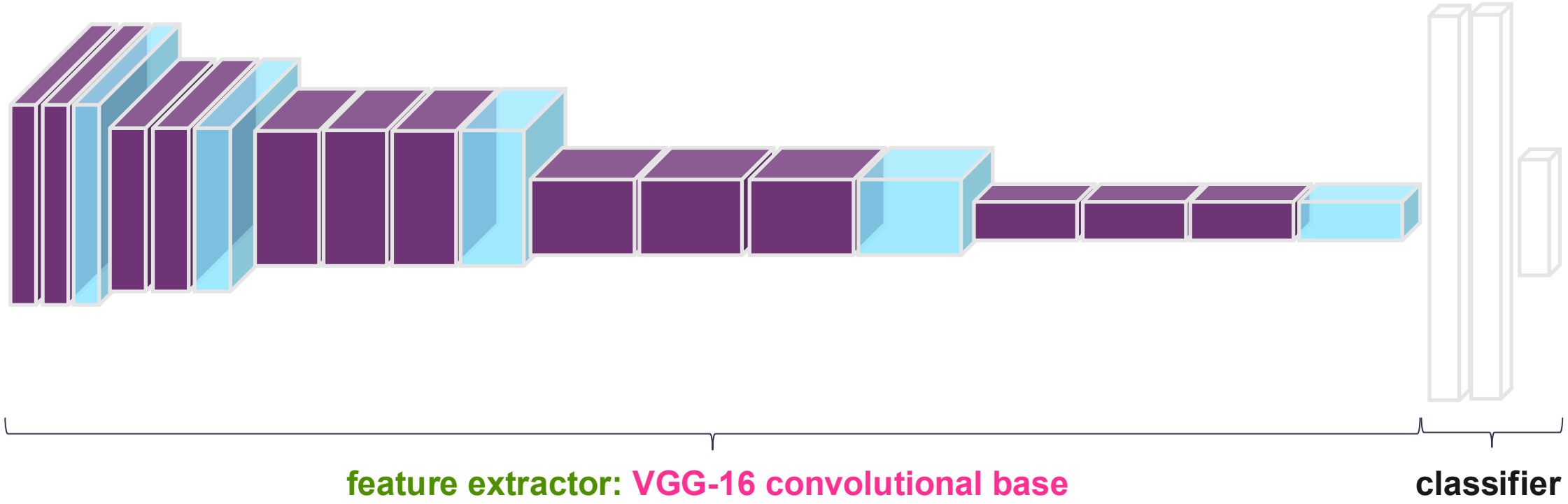



https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Flatten layer
Dense layer
Output layer

# Transfer learning

## Pretrained model as a feature extractor

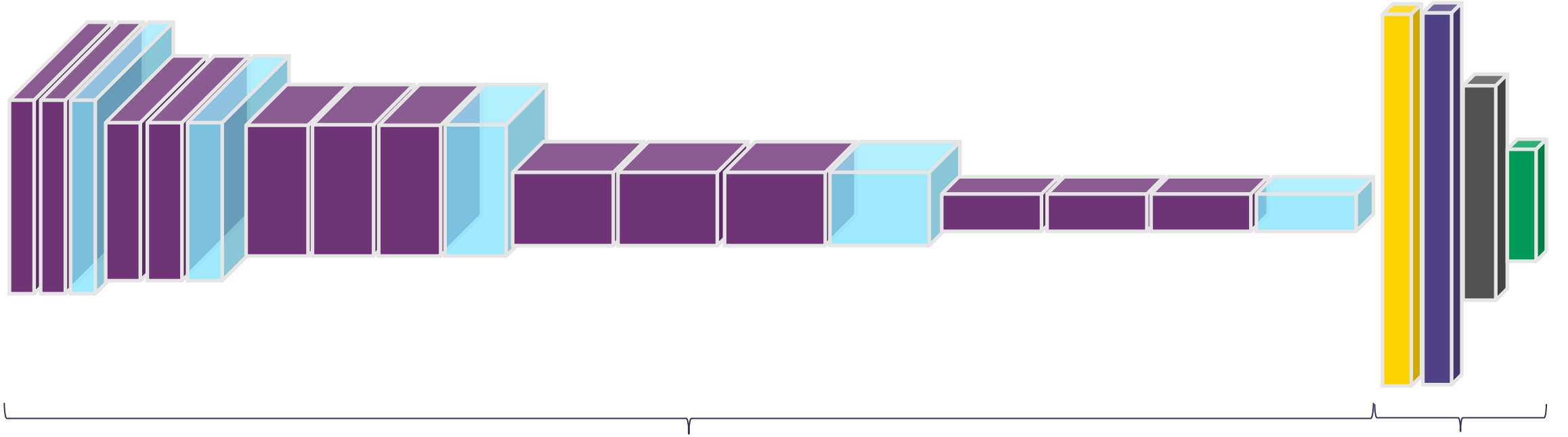



https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Flatten layer
Dense layer
Output layer

# Transfer learning

## Pretrained model as a feature extractor

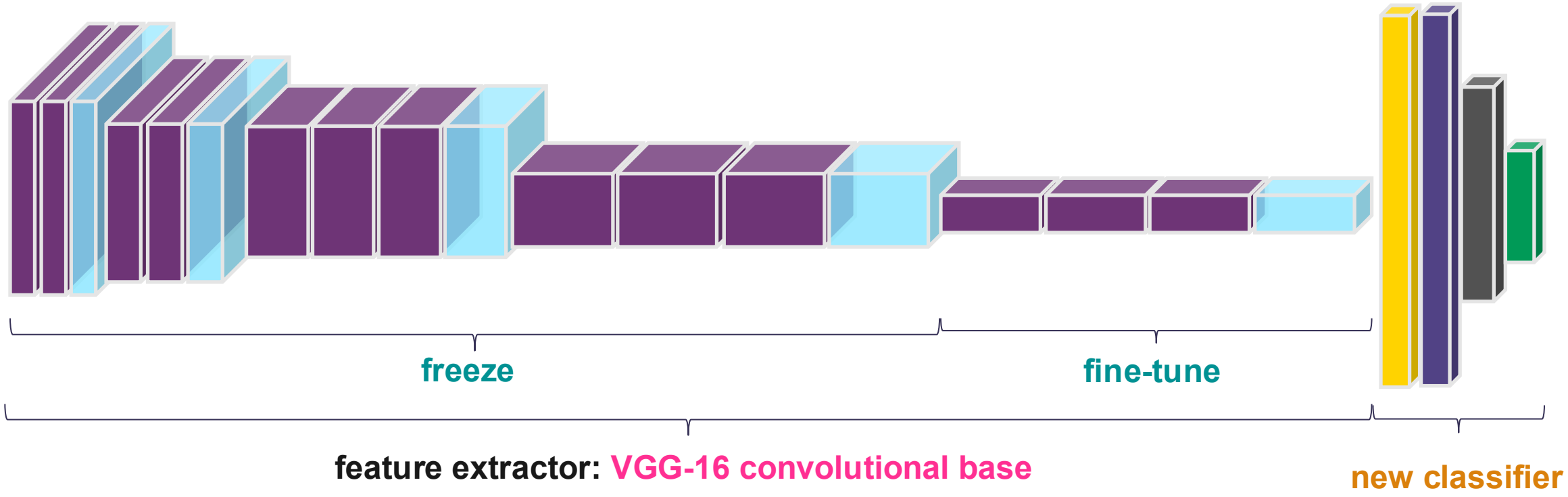



https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

# Transfer learning

## Fine-tuning a pretrained model




https://arxiv.org/pdf/1409.1556

Conv2D layer
Max pooling layer
Dropout layer
Flatten layer
Dense layer
Output layer

# Exercise 4:
# Food Classification

# Regularization

## Exercise 4

### INSTRUCTIONS

1. Create a CNN that classifies 🍽 food images containing 10 classes. The dataset that we will use is a subset of the Food-101 dataset from Kaggle, which contains labeled food images.

2. The data preparation and creation of a baseline CNN is already provided. Improve the performance of the baseline model by applying the regularization techniques discussed.

# Reminders

# Reminders

## Next Session

Session 7 and 8 (April 30, 2026) – CNN, RNN

## Deliverables

**Due this week**

- R03 - Post-Class Reflection 3 (due EOD April 24, 2026)

**Due next, next week**

- E4 – Exercise 4 (due EOD May 4, 2026)

ASIAN INSTITUTE OF MANAGEMENT

Machine Learning 3

# Convolutional Neural Networks

## Sessions 5-6

MSDS 2026

23 April 2026

ASIAN INSTITUTE OF MANAGEMENT